

Data Lab 报告

姓名：林泽远

学号：2025201835

测试结果

总分	37/37	
bitAnd	1/1	4
bitXor	1/1	7
samesign	2/2	9
logtwo	4/4	24
byteSwap	4/4	11
reverse	3/3	23
logicalShift	3/3	6
leftBitCount	4/4	27
float_i2f	4/4	30
floatScale2	4/4	13
float64_f2i	3/3	15
floatPower2	4/4	9

```
$ cd ~/datalab2026 && python3 test.py -V
```

```
bitAnd      1/1:    PASS    Pass using 4 operations
bitXor      1/1:    PASS    Pass using 7 operations
samesign    2/2:    PASS    Pass using 9 operations
logtwo      4/4:    PASS    Pass using 24 operations
byteSwap    4/4:    PASS    Pass using 11 operations
reverse     3/3:    PASS    Pass using 23 operations
logicalShift 3/3:    PASS    Pass using 6 operations
leftBitCount 4/4:    PASS    Pass using 27 operations
float_i2f    4/4:    PASS    Pass using 30 operations
floatScale2 4/4:    PASS    Pass using 13 operations
float64_f2i 3/3:    PASS    Pass using 15 operations
floatPower2 4/4:    PASS    Pass using 9 operations
```

```
Total points: 37
```

基础位运算

bitAnd

利用德摩根律： $x \& y = \sim(\sim x \mid \sim y)$ 。

bitXor

$x \wedge y = \sim(\sim x \& \sim y) \& \sim(x \& y)$ 。前半部分表示至少一个数为 1，后半部分排除两个数同时为 1。

samesign

先通过算术右移取得符号位，若符号相同则结果为 1；再用 $!x \wedge !y$ 排除一个为 0、另一个非 0 的情况。

logtwo

按 16、8、4、2、1 的步长做二分判断，确定最高有效位所在区间并累计指数。

byteSwap

分别取出两个字节 a 和 b，计算 $\text{diff} = a \wedge b$ 。再将 diff 分别异或回两个字节位置，实现交换。

reverse

按 1、2、4、8、16 位粒度逐层交换相邻位组，五轮后完成 32 位反转。

整数与浮点转换

logicalShift

算术右移会补充符号位，因此右移后构造掩码清除高位补充的 1。掩码写作 $\sim((1 \ll 31) \gg n \ll 1)$ ，再与右移结果相与。

leftBitCount

令 $y = \sim x$ ，统计 y 开头连续 0 的个数。依次检查 16、8、4、2、1 位：若高位全为 0，则累加该位宽并左移跳过。 $x = -1$ 时最后用 $!\sim x$ 补足到 32。

float_i2f

分离符号与绝对值，找到最高有效位 e 。 $e \leq 23$ 时直接左移构造尾数；否则截取高 23 位并计算舍入余数。向偶数舍入条件合并为 $\text{rem} \geq \text{half} + !(m \& 1)$ 。尾数进位时清零尾数并增加指数。

浮点函数

floatScale2

exp = 0xFF 时直接返回 Inf 或 NaN；exp = 0 时用 uf + (uf & 0x7FFFFFFF) 将非规格化尾数乘 2；普通数指数加一，若溢出则保留符号并返回无穷大。

float64_f2i

从 uf2 中提取符号和 11 位指数，再构造双精度有效数字高 32 位：val = 0x800000000u | ((uf2 & 0xFFFFF) << 11) | (uf1 >> 21)。指数小于 1023 返回 0，大于 1054 返回 0x800000000；其余按 1054 - exp 右移实现向零取整，并检查有符号整数范围。

floatPower2

x > 127 返回正无穷 0x7F800000；x < -149 返回 0；-149 <= x <= -127 返回非规格化位 1 << (x + 149)；-126 <= x <= 127 返回规格化位 (x + 127) << 23。